TROUBLESHOOTING

Error reference

Look up Claude Code runtime error messages with what each one means and how to fix it.

This page lists runtime errors Claude Code displays and how to recover from each one, plus what to check when responses seem off without an error. For installation errors such as `command not found` or TLS failures during setup, see **Troubleshoot installation and login**.

These errors and recovery commands apply across the CLI, the **Desktop app**, and **Claude Code on the web**, since all three wrap the same Claude Code CLI. For surface-specific issues, see the troubleshooting section on that surface's page.

> Claude Code calls the Claude API for model responses, so most runtime errors map to an underlying API error code. This page covers what each error means inside Claude Code and how to recover. For the raw HTTP status code definitions, see the **Claude Platform error reference**.

Find your error

Match the message you see in your terminal to a section below.

Message	Section
`API Error: 500 Internal server error`	**Server errors**
`API Error: Repeated 529 Overloaded errors`	**Server errors**
`Request timed out`	**Server errors**, or **Network** if the message mentions your internet connection
`<model> is temporarily unavailable, so auto mode cannot determine the safety of...`	**Server errors**
`Auto mode could not evaluate this action and is blocking it for safety`	**Server errors**
`Auto mode classifier transcript exceeded context window`	**Server errors**
`You've hit your session limit` / `You've hit your weekly`	**Usage limits**

Message	Section
`limit`	
`Usage credits required for 1M context`	**Usage limits**
`Server is temporarily limiting requests`	**Usage limits**
`Request rejected (429)`	**Usage limits**
`Credit balance is too low`	**Usage limits**
`Not logged in · Please run /login`	**Authentication**
`Could not resolve authentication method`	**Authentication**
`Invalid API key`	**Authentication**
`This organization has been disabled`	**Authentication**
`Your organization has disabled API key authentication`	**Authentication**
`Your organization has disabled Claude subscription access`	**Authentication**
`Routines are disabled by your organization's policy`	**Authentication**
`OAuth token revoked` / `OAuth token has expired`	**Authentication**
`does not meet scope requirement user:profile`	**Authentication**
`Unable to connect to API`	**Network**
`Waiting for API response · will retry in`	**Automatic retries**, or **Network** if it persists
`SSL certificate verification failed`	**Network**
`403` with `x-deny-reason: host_not_allowed` in a cloud or routine session	**Network**
`Prompt is too long`	**Request errors**
`Error during compaction: Conversation too long`	**Request errors**
`Request too large`	**Request errors**
`Image was too large`	**Request errors**
`Unable to resize image`	**Request errors**
`PDF too large` / `PDF is password protected`	**Request errors**
`Extra inputs are not permitted`	**Request errors**
`There's an issue with the selected model`	**Request errors**
`Claude Opus is not available with the Claude Pro plan`	**Request errors**

Message	Section
`Model ... is restricted by your organization's settings`	**Request errors**
`thinking.type.enabled is not supported for this model`	**Request errors**
`max_tokens must be greater than thinking.budget_tokens`	**Request errors**
`API Error: 400 due to tool use concurrency issues`	**Request errors**
`Claude Code is unable to respond to this request, which appears to violate our Usage Policy`	**Request errors**
Responses seem lower quality than usual	**Response quality**

Automatic retries

Claude Code retries transient failures before showing you an error. Server errors, overloaded responses, request timeouts, temporary 429 throttles, and dropped connections are all retried up to 10 times with exponential backoff. While retrying, the spinner shows a `Retrying in Ns · attempt x/y` countdown.

If no data arrives on the response stream for 20 seconds while a request is still pending, the spinner shows `Waiting for API response · will retry in … · check your network` before any retry has started. The request has not failed yet: the countdown runs to the point where Claude Code aborts the stalled connection and retries, so the banner clears on its own once data resumes or the retry succeeds. As of v2.1.185 the threshold is 20 seconds; earlier versions show the banner after 10 seconds with different wording. If it reappears on every attempt, treat it as a **network issue**.

When you see one of the errors on this page, those retries have already been exhausted. You can tune the behavior with these environment variables:

Variable	Default	Effect
`CLAUDE_CODE_MAX_RETRIES`	10	Number of retry attempts. Capped at 15 as of v2.1.186. Lower it to surface failures faster in scripts.
`CLAUDE_CODE_RETRY_WATCHDOG`	unset	Set to `1` in unattended sessions such as CI jobs to retry `429` and `529` capacity errors indefinitely instead of failing after `CLAUDE_CODE_MAX_RETRIES` attempts.
`API_TIMEOUT_MS`	600000	Per-request timeout in milliseconds. Raise it for slow networks or proxies.

Server errors

These errors come from the inference provider rather than your account or request. On the Anthropic API that means Anthropic infrastructure. On Bedrock, Vertex AI, Foundry, or a custom gateway it means that provider's infrastructure.

API Error: 500 Internal server error

Claude Code shows the status code and the API's error message for any 5xx response. The example below shows a 500 response on the Anthropic API:

```
API Error: 500 Internal server error. This is a server-side
issue, usually temporary — try again in a moment. If it persists,
check https://status.claude.com.
```

The trailing sentence names where to check service health and varies by provider. Bedrock, Vertex AI, and Foundry configurations name that provider's service status. A custom `ANTHROPIC_BASE_URL` names the gateway host.

This indicates an unexpected failure inside the API. It is not caused by your prompt, settings, or account.

What to do:

- Check **status.claude.com**, or the provider status page named in the message, for active incidents
- Wait a minute, then send your message again. Your original message is still in the conversation, so for a long prompt you can type `try again` instead of pasting the whole thing.
- If the error persists with no posted incident, run `/feedback` so Anthropic can investigate with your request details. See **Report an error** if `/feedback` is unavailable in your environment.

API Error: Repeated 529 Overloaded errors

The API is temporarily at capacity across all users. Claude Code has already retried several times before showing this message:

```
API Error: Repeated 529 Overloaded errors. The API is at capacity
— this is usually temporary. Try again in a moment. If it
persists, check https://status.claude.com.
```

The trailing sentence varies by provider in the same way as the 500 error above. A 529 is not your usage limit and does not count against your quota.

What to do:

- Check **status.claude.com**, or the provider status page named in the message, for capacity notices
- Try again in a few minutes
- Run `/model` and switch to a different model to keep working, since capacity is tracked per model. Claude Code prompts you to do this when one model is under particularly high load, for example `Opus is experiencing high load, please use /model to switch to Sonnet`.

Request timed out

The API did not respond before the connection deadline.

```
Request timed out
```

This can happen during periods of high load or when a very large response is being generated. The default request timeout is 10 minutes.

What to do:

- Retry the request
- For long-running tasks, break the work into smaller prompts
- If a slow network or proxy is the cause, raise `API_TIMEOUT_MS` as described in **Automatic retries**
- If timeouts are frequent and your network is otherwise healthy, see **Network and connection errors** below

Auto mode cannot determine the safety of an action

The model that **auto mode** uses to classify actions could not produce a decision, so auto mode did

not approve the action automatically. The message you see depends on why the classifier failed.

Reads, searches, and edits inside your working directory skip the classifier, so they keep working in all of these cases.

When the classifier model is overloaded:

```
<model> is temporarily unavailable, so auto mode cannot determine
the safety of <tool> right now. Wait briefly and then try this
action again.
```

What to do:

- Retry after a few seconds; Claude sees the same message and usually retries on its own
- If retries keep failing, continue with read-only tasks and come back to the blocked action later
- This is transient and unrelated to **auto mode eligibility**; you do not need to change settings

When the classifier returned an unparseable response:

```
Auto mode could not evaluate this action and is blocking it for
safety — run with --debug for details
```

What to do:

- Retry the action; this usually succeeds on the next attempt
- Run `claude --debug` and repeat the action to see the underlying classifier response in the debug log

When a separate API safety check blocked the classifier request because of earlier conversation content:

```
Auto mode could not evaluate this action and is blocking it for
safety — a safety check separate from auto mode blocked this
request because of earlier conversation content — it isn't about
the action itself — run with --debug for details
```

What to do:

- This is not a decision about your action. A safety filter on the API was triggered by existing content in your conversation when auto mode sent the conversation to the classifier
- Retrying will not help; the same conversation content will trigger the filter again
- Switch to a different **permission mode** so you can approve the action when prompted, or start a fresh conversation without the triggering content

When the conversation has grown larger than the classifier's context window:

```
Auto mode classifier transcript exceeded context window — falling
back to manual approval (try /compact to reduce conversation
size)
```

In an interactive session, auto mode falls back to a normal permission prompt for that action so you can approve or deny it manually. In **non-interactive mode** the run aborts because the transcript only grows and retrying cannot succeed.

What to do:

- Approve or deny the action in the prompt that appears
- Run `/compact` to reduce the conversation size so subsequent actions fit within the classifier window again

Usage limits

These errors mean a quota tied to your account or plan has been reached. They are distinct from **server errors**, which affect everyone.

You've hit your session limit

Subscription plans include a rolling usage allowance. When it runs out you see one of these messages:

```
You've hit your session limit · resets 3:45pm
You've hit your weekly limit · resets Mon 12:00am
You've hit your Opus limit · resets 3:45pm
```

Claude Code blocks further requests until the reset time shown in the message.

What to do:

- Wait for the reset time shown in the error
- Run `/usage` to see your plan limits and when they reset
- Run `/usage-credits` to buy additional usage on Pro and Max, or to request it from your admin on Team and Enterprise. See **usage credits for paid plans** for how this is billed.
- To upgrade your plan for higher base limits, see **claude.com/pricing**

To watch your remaining allowance before you hit the limit, add the `rate_limits` fields to a **custom status line**, or in the Desktop app click the **usage ring** next to the model picker.

Usage credits required for 1M context

The selected model uses the 1M-token extended context window, and your plan only includes it through usage credits.

```
API Error: Usage credits required for 1M context · run /usage-
credits to turn them on, or /model to switch to standard context
```

This is an entitlement check, not a quota exhaustion. It fires even when your session and weekly allowances have capacity remaining. See **Extended context** for which plans include 1M context directly and which require usage credits.

When this error appears mid-conversation because the context grew past 200K tokens, Claude Code automatically compacts the conversation back under the standard context limit and keeps the session at that limit afterward, so no action is needed. On versions before v2.1.172, the error repeated on every subsequent request including `/compact`; run `/clear` on those versions to recover. The steps below apply when you explicitly selected a `[1m]` model.

What to do:

- Run `/model` and select the variant without the `[1m]` suffix to fall back to the standard context window
- Run `/usage-credits` to turn on metered billing for the 1M variant on Pro and Max, or to request it from your admin on Team and Enterprise
- If the error persists after `/model`, a 1M model ID may be set elsewhere. See **There's an issue with the selected model** for the configuration locations to check in priority order.

- To remove 1M variants from the model picker entirely, set `CLAUDE_CODE_DISABLE_1M_CONTEXT=1`

Server is temporarily limiting requests

The API applied a short-lived throttle that is unrelated to your plan quota.

```
API Error: Server is temporarily limiting requests (not your
usage limit)
```

This is **retried automatically** before being shown.

What to do:

- Wait briefly and try again
- Check **status.claude.com** if it persists

Request rejected (429)

You have hit the rate limit configured for your API key, Amazon Bedrock project, or Google Vertex AI project.

```
API Error: Request rejected (429) · this may be a temporary
capacity issue. If it persists, check https://status.claude.com.
```

The trailing sentence names where to check service health and varies by provider. Bedrock, Vertex AI, and Foundry configurations name that provider's service status instead of the Anthropic status page. A custom `ANTHROPIC_BASE_URL` names the gateway host.

What to do:

- Run `/status` and confirm the active credential is the one you expect. A stray `ANTHROPIC_API_KEY` in your environment can route requests through a low-tier key instead of your subscription.
- Check your provider console for the active limits and request a higher tier if needed
- For Anthropic API keys, see the **rate limits reference** for how tiers work and how to set per-workspace caps
- Reduce concurrency: lower `CLAUDE_CODE_MAX_TOOL_USE_CONCURRENCY`, avoid running many

parallel subagents, or switch to a smaller model with `/model` for high-volume scripted runs

Credit balance is too low

Your Console organization has run out of prepaid credits.

```
Credit balance is too low
```

What to do:

- Add credits at **platform.claude.com/settings/billing**, and consider enabling auto-reload there so the balance refills before it hits zero
- Switch to subscription authentication with `/login` if you have a Pro, Max, Team, or Enterprise plan
- Set per-workspace spend caps in the Console to prevent a single project from draining the org balance. See **Manage costs effectively**.

Authentication errors

These errors mean Claude Code cannot prove who you are to the API. Run `/status` at any time to see which credential is currently active.

Not logged in

No valid credential is available for this session.

```
Not logged in · Please run /login
```

What to do:

- Run `/login` to authenticate with your Claude subscription or Console account
- If you expected an environment variable to authenticate you, confirm `ANTHROPIC_API_KEY` is set and exported in the shell where you launched `claude`
- For CI or automation where interactive login is not possible, configure an `apiKeyHelper` script that fetches a key at startup

- See **Authentication precedence** to understand which credential wins when several are present

If you are prompted to log in repeatedly, see **Not logged in or token expired** for system clock and macOS Keychain fixes.

Could not resolve authentication method

The session reached the API client without any credential. This appears in **background sessions**, cloud sessions, and Agent SDK contexts where the interactive login check does not run before the first request.

```
Could not resolve authentication method. Expected one of apiKey,
authToken, credentials, config, or profile to be set. Or for one
of the "X-Api-Key" or "Authorization" headers to be explicitly
omitted
```

Before v2.1.174, a background or cloud session assigned to an idle pre-initialized worker could fail this way even when valid credentials were configured. Upgrade to recover. On current versions the error means no credential was available to the worker process.

What to do:

- Upgrade to v2.1.174 or later if this appears in a background or cloud session and your credentials are already configured
- Confirm `ANTHROPIC_API_KEY`, `CLAUDE_CODE_OAUTH_TOKEN`, or your cloud provider credentials are set in the environment that launches the worker, not only in your interactive shell
- For the Agent SDK, see **authentication setup**
- Run `/status` in an interactive session in the same environment to confirm which credential source resolves

Invalid API key

The `ANTHROPIC_API_KEY` environment variable or `apiKeyHelper` script returned a key the API rejected.

```
Invalid API key · Fix external API key
```

What to do:

- Check for typos and confirm the key has not been revoked in the **Console**
- Run `env | grep ANTHROPIC` in the same shell. Tools like direnv, dotenv shell plugins, and IDE terminals can load a stale key from a `.env` file in your project without you setting it explicitly.
- Unset `ANTHROPIC_API_KEY` and run `/login` to use subscription auth instead
- If the key comes from an `apiKeyHelper` script, run the script directly to confirm it prints a valid key on stdout
- Run `/status` to confirm which credential source Claude Code is actually using

This organization has been disabled

A stale `ANTHROPIC_API_KEY` from a disabled Console organization is overriding your subscription login.

```
Your ANTHROPIC_API_KEY belongs to a disabled organization · Unset
the environment variable to use your other credentials
API Error: 400 ... This organization has been disabled.
```

Environment variables take precedence over `/login`, so a key exported in your shell profile or loaded from a `.env` file is used even when you have a working Pro or Max subscription. In non-interactive mode (`-p`), the key is always used when present.

What to do:

- Unset `ANTHROPIC_API_KEY` in the current shell and remove it from your shell profile, then relaunch `claude`
- Run `/status` afterward to confirm the active credential is your subscription
- If no environment variable is set and the error persists, the disabled organization is the one tied to your `/login`. Contact support or sign in with a different account.

Your organization has disabled API key authentication

Your Console organization's admin has turned off API key authentication, so the API rejects the key Claude Code is sending. The recovery hint after the `·` varies by where the key came from:

```
Your organization has disabled API key authentication · Run
/login to sign in with your claude.ai account
Your organization has disabled API key authentication · Unset
ANTHROPIC_API_KEY to use your claude.ai account instead
Your organization has disabled API key authentication · Unset
ANTHROPIC_API_KEY and run /login to sign in with your claude.ai
account
Your organization has disabled API key authentication · Unset the
apiKeyHelper setting and run /login to sign in with your
claude.ai account
```

Environment variables and `apiKeyHelper` take precedence over `/login`, so running `/login` alone does not help while either is still supplying a key. See **Authentication precedence**.

What to do:

- If the message names `ANTHROPIC_API_KEY`, unset it in the current shell and remove it from your shell profile or `.env` file, then relaunch `claude`
- If the message names `apiKeyHelper`, remove the `apiKeyHelper` setting from your `settings.json`
- Run `/login` to sign in with your claude.ai account
- Run `/status` afterward to confirm the active credential is your subscription rather than an API key
- If you need API key authentication for automation, ask your organization admin to re-enable it in the Console

Your organization has disabled Claude subscription access

Your Claude organization does not allow signing in to Claude Code with a subscription login. Running `/login` again with the same account returns the same error.

```
Your organization has disabled Claude subscription access for
Claude Code · Use an Anthropic API key instead, or ask your admin
to enable access
```

This is a server-side organization setting, so it cannot be overridden from local settings, environment variables, or CLI flags. The Agent SDK and `-p` non-interactive mode surface this as the

`oauth_org_not_allowed` error code.

What to do:

- Ask your admin to enable Claude Code access for your organization
- Authenticate with a Console API key instead of your subscription. See **Claude Console authentication** for setup.
- If you are the admin and do not see an option to enable access, contact **Anthropic support**

Routines are disabled by your organization's policy

An Owner in your Team or Enterprise organization has turned off routines at the organization level. The error appears when you try to create or run a routine, including from `/schedule` and the **Routines** UI on claude.ai/code.

```
Routines are disabled by your organization's policy.
```

This is a server-side setting, so it cannot be overridden from local settings, environment variables, or CLI flags.

What to do:

- Ask an Owner in your organization to enable the **Routines** toggle at **claude.ai/admin-settings/claude-code**
- For one-off scheduled work that does not require organization-level routines, see **scheduled tasks**

OAuth token revoked or expired

Your saved login is no longer valid. A revoked token means you signed out everywhere or an admin removed access; an expired token means the automatic refresh failed mid-session.

```
OAuth token revoked · Please run /login
OAuth token has expired · Please run /login
API Error: 401 ... authentication_error
```

What to do:

- Run `/login` to sign in again
- If the error returns within the same session after re-authenticating, run `/logout` first to fully clear the stored token, then `/login`
- For repeated prompts to log in across launches, see the system clock and macOS Keychain checks in **Troubleshooting**
- For other failures including `403 Forbidden` and OAuth browser issues, see **Login and authentication**

OAuth scope requirement

The stored token predates a permission scope that a newer feature needs. You see this most often from `/usage` and the status line usage indicator:

```
OAuth token does not meet scope requirement: user:profile
```

What to do:

- Run `/login` to mint a new token with the current scopes. You do not need to log out first.

Network and connection errors

These errors mean a network request from Claude Code failed to reach its destination. They usually originate in your local network, proxy, or firewall, or in the cloud environment's network policy.

Unable to connect to API

The TCP connection to the API failed or never completed.

```
Unable to connect to API. Check your internet connection
Unable to connect to API (ECONNREFUSED)
Unable to connect to API (ECONNRESET)
Unable to connect to API (ETIMEDOUT)
fetch failed
Request timed out. Check your internet connection and proxy
settings
```

Common causes include no internet access, a VPN that blocks `api.anthropic.com`, or a required corporate proxy that is not configured.

What to do:

- Confirm you can reach the API host from the same shell by running `curl -I https://api.anthropic.com`. On Windows PowerShell use `curl.exe -I https://api.anthropic.com` so the built-in `Invoke-WebRequest` alias is not used.
- If you are behind a corporate proxy, set `HTTPS_PROXY` before launching Claude Code and see **Network configuration**
- If you route through an LLM gateway or relay, set `ANTHROPIC_BASE_URL` to its address. See **Connect Claude Code to an LLM gateway** for setup.
- Ensure your firewall allows the hosts listed in **Network access requirements**
- Intermittent failures are **retried automatically**; persistent failures point to a local network issue

If `curl` succeeds but Claude Code still fails, the cause is usually something between the runtime and the network rather than the network itself:

- On Linux and WSL, check `/etc/resolv.conf` for an unreachable nameserver. WSL in particular can inherit a broken resolver from the host.
- On macOS, a VPN client that was disconnected or uninstalled can leave a tunnel interface or routing rule behind. Check `ifconfig` for stale `utun` interfaces and remove the VPN's network extension in System Settings.
- Docker Desktop and similar container runtimes can intercept outbound traffic. Quit them and retry to rule this out.

SSL certificate errors

A proxy or security appliance on your network is intercepting TLS traffic with its own certificate, and Claude Code does not trust it.

```
Unable to connect to API: SSL certificate verification failed.
Check your proxy or corporate SSL certificates
Unable to connect to API: Self-signed certificate detected
```

What to do:

- Export your organization's CA bundle and point Claude Code at it with `NODE_EXTRA_CA_CERTS=/path/to/ca-bundle.pem`
- See **Network configuration** for full setup instructions
- Do not set `NODE_TLS_REJECT_UNAUTHORIZED=0`, which disables certificate validation entirely

Host not allowed in a cloud session

An outbound HTTP request from a cloud session or routine was blocked by the environment's network policy.

```
HTTP 403
x-deny-reason: host_not_allowed
```

You may also see a TLS certificate that doesn't match the destination's real certificate. The cloud environment routes outbound traffic through a proxy that enforces the network policy, so a mismatched certificate means the proxy terminated the connection, not the destination.

This is not a client-side network problem. Cloud sessions and **routines** run inside a sandboxed environment whose outbound traffic is filtered to the environment's allowlist. The **Default** environment uses **Trusted** access, which permits the **default allowlist** of package registries, cloud provider APIs, container registries, and common development domains but blocks everything else.

What to do:

- Open the routine for editing, or start a cloud session. Select the cloud icon showing your environment's name, such as **Default**, to open the selector. Hover over your environment and click the settings icon.
- In the **Update cloud environment** dialog, change **Network access** from **Trusted** to **Custom**, then add the blocked domain to **Allowed domains**. Enter one domain per line. Check **Also include default list of common package managers** to keep the **default allowlist** alongside your custom domains. Select **Full** instead if you want unrestricted access.
- Click **Save changes**. The next run uses the updated allowlist.

See **Network access** for access levels and the default allowlist. Local CLI sessions are not affected by this policy.

Request errors

These errors mean the API received your request but rejected its content.

Prompt is too long

The conversation plus attached files exceeds the model's context window.

```
Prompt is too long
```

What to do:

- Run `/compact` to summarize earlier turns and free space, or `/clear` to start fresh
- Run `/context` to see a breakdown of what is consuming the window: system prompt, tools, memory files, and messages
- Disable MCP servers you are not using with `/mcp disable <name>` to remove their tool definitions from context
- Trim large `CLAUDE.md` memory files, or move instructions into **path-scoped rules** that load only when relevant
- Subagents inherit every MCP tool definition from the parent session, which can fill their context window before the first turn. Disable MCP servers you are not using before spawning subagents.
- Auto-compact is on by default and normally prevents this error. If you have set `DISABLE_AUTO_COMPACT`, re-enable it or run `/compact` manually before the window fills.

See **Explore the context window** for an interactive view of how context fills up.

Error during compaction: Conversation too long

`/compact` itself failed because there is not enough free context to hold the summary it produces.

```
Error during compaction: Conversation too long. Press esc twice
to go up a few messages and try again.
```

This can happen when the window is already full at the moment auto-compact triggers, or when you run `/compact` after seeing `Prompt is too long`.

What to do:

- Press Esc twice to open the message list and step back several turns. This drops the most recent messages from context. Then run `/compact` again.
- If stepping back does not free enough space, run `/clear` to start a fresh session. Your previous conversation is preserved and can be reopened with `/resume`.

Request too large

The raw request body exceeded the API's byte limit before tokenization, usually because of a large pasted file or attachment.

```
Request too large (max 30 MB). Double press esc to go back and
remove or shrink the attached content.
```

This is a size limit on the HTTP request, separate from the **context window limit**.

What to do:

- Press Esc twice and step back past the turn that added the oversized content
- Reference large files by path instead of pasting their contents, so Claude can read them in chunks
- For images, see **Image was too large** below

Image was too large

A pasted or attached image exceeds the API's size or dimension limits.

```
Image was too large. Double press esc to go back and try again
with a smaller image.
API Error: 400 ... image dimensions exceed max allowed size
```

Claude Code replaces the unprocessable image with a text placeholder and retries, so subsequent messages succeed. On versions before 2.1.142, a pasted image could remain in the conversation and repeat the same error on every subsequent message. To recover on those versions, press Esc twice and step back past the turn where the image was added.

What to do:

- Resize the image before pasting. The API accepts images up to 8000 pixels on the longest edge for a single image, or 2000 pixels when many images are in context.
- Take a tighter screenshot of the relevant region instead of the full screen

Unable to resize image

Claude Code could not downscale an attached image before sending it to the API.

```
Unable to resize image — image processing is unavailable and
dimensions could not be read from the file header. Please convert
the image to PNG, JPEG, GIF, or WebP.
Unable to resize image — dimensions exceed the 2000x2000px limit
and image processing failed. Please resize the image to reduce
its pixel dimensions.
Unable to resize image (… raw, … base64). The image exceeds the …
API limit and compression failed. Please resize the image
manually or use a smaller image.
Unable to resize image — could not verify image dimensions are
within the 2000x2000px API limit.
```

Claude Code normally resizes large images automatically. These errors mean the native image processor failed to load or returned an error, so the image could not be resized to fit within API limits.

What to do:

- If the message asks you to convert the image, convert it to PNG, JPEG, GIF, or WebP and attach it again. Claude Code can verify dimensions for these formats without the image processor.
- If the message reports a dimension or size limit, resize or recompress the image below that limit before attaching.

PDF errors

The PDF you attached could not be processed.

```
PDF too large (max 100 pages, 32 MB). Try splitting it or
extracting text first.
PDF is password protected. Try removing protection or extracting
text first.
The PDF file was not valid. Try converting to a different format
first.
```

What to do:

- For oversized PDFs, ask Claude to read a page range with the Read tool instead of attaching the whole file, or extract text with a tool like `pdftotext` and reference the output file by path
- For protected or invalid PDFs, remove the password or re-export the file from its source application, then try again

Extra inputs are not permitted

A proxy or LLM gateway between Claude Code and the API stripped the `anthropic-beta` request header, so the API rejected fields that depend on it.

```
API Error: 400 ... Extra inputs are not permitted ...
context_management
API Error: 400 ... Extra inputs are not permitted ...
tools.0.custom.input_examples
API Error: 400 ... Unexpected value(s) for the `anthropic-beta`
header
```

Claude Code sends beta-only fields such as `context_management`, `effort`, and tool `input_examples` alongside an `anthropic-beta` header that enables them. When a gateway forwards the body but drops the header, the API sees fields it does not recognize.

What to do:

- Configure your gateway to forward the `anthropic-beta` header. See **feature pass-through** for what gateways must forward.
- As a fallback, set `CLAUDE_CODE_DISABLE_EXPERIMENTAL_BETAS=1` before launching. This disables features that require the beta header so requests succeed through a gateway that cannot forward it.

There's an issue with the selected model

The configured model name was not recognized or your account lacks access to it. As of v2.1.160 the trailing hint, shown here in its interactive form, varies by surface.

```
There's an issue with the selected model (claude-...). It may not
exist or you may not have access to it. Run /model to pick a
different model.
```

What to do:

- **Interactive CLI**: run `/model` to pick from models available to your account.
- **Non-interactive mode (`-p`)**: pass `--model` with a valid alias or ID, or set `ANTHROPIC_MODEL`. The error text shows `Run --model` on this surface.
- **Agent SDK**: the error text omits the hint because the model is set programmatically. Set `model` on `Options` in TypeScript or `ClaudeAgentOptions(model=...)` in Python, and handle the structured `model_not_found` error to surface your own retry or model picker.
- Use an alias such as `sonnet` or `opus` instead of a full versioned ID. Aliases resolve to a maintained default so they do not go stale. See **Model configuration**.
- If the wrong model keeps coming back in the CLI, a stale ID is set somewhere. Check in **priority order**: the `--model` flag, the `ANTHROPIC_MODEL` environment variable, then the `model` field in `.claude/settings.local.json`, your project's `.claude/settings.json`, and `~/.claude/settings.json`. Remove the stale value and Claude Code falls back to your account default.
- For Vertex AI deployments, see **Vertex AI troubleshooting**.

Claude Opus is not available with the Claude Pro plan

Your active subscription plan does not include the model you selected.

```
Claude Opus is not available with the Claude Pro plan · Select a
different model in /model
```

What to do:

- Run `/model` and select a model your plan includes

- If you upgraded your plan recently and still see this, run `/logout` then `/login`. The stored token reflects your plan at the time you signed in, so upgrading on the web does not take effect in an existing session until you re-authenticate.
- See **claude.com/pricing** for which models each plan includes

Model is restricted by your organization's settings

Your organization admin has disabled this model in the Claude Console, or it is excluded by an `availableModels` allowlist in managed settings. When the restricted model was set with `--model`, `ANTHROPIC_MODEL`, or the `model` setting, Claude Code substitutes an allowed model and continues. Typing `/model <name>` for a restricted model is rejected with `Run /model to choose a different model.` and the session keeps its current model.

```
Model "claude-opus-4-8" is restricted by your organization's
settings. Using claude-sonnet-4-6 instead.
```

What to do:

- Run `/model` to pick from the models your organization allows. Restricted models are hidden from the picker.
- If the restricted model was set in `--model`, `ANTHROPIC_MODEL`, or the `model` field of a settings file, remove or update that value so the notice does not recur on each launch
- If you need access to the restricted model, ask your organization admin to enable it. See **Organization model restrictions**.

thinking.type.enabled is not supported for this model

Your Claude Code version is older than the minimum for Opus 4.7 or Opus 4.8. The CLI sent a thinking configuration the model no longer accepts.

```
API Error: 400 ... "thinking.type.enabled" is not supported for
this model. Use "thinking.type.adaptive" and
"output_config.effort" to control thinking behavior.
```

What to do:

- Run `claude update` and restart Claude Code. Opus 4.7 needs v2.1.111 or later. Opus 4.8 needs

v2.1.154 or later

- If you cannot upgrade, run `/model` and select Opus 4.6 or Sonnet instead
- If you hit this in the **Agent SDK**, upgrade the SDK package instead. Opus 4.8 needs TypeScript SDK v0.3.154 or later and Python SDK v0.2.88 or later

Thinking budget exceeds output limit

The configured extended thinking budget exceeds the maximum response length, so there is no room left for the actual answer.

```
API Error: 400 ... max_tokens must be greater than
thinking.budget_tokens
```

Claude Code adjusts these values automatically on the Anthropic API. You typically see this error on Amazon Bedrock or Google Vertex AI when `MAX_THINKING_TOKENS` is set higher than the provider's output limit, or when plan mode raises the thinking budget.

What to do:

- Lower `MAX_THINKING_TOKENS`, or raise `CLAUDE_CODE_MAX_OUTPUT_TOKENS` above the thinking budget
- See **Extended thinking** for how the budget interacts with output length

Tool use or thinking block mismatch

The conversation history reached the API in an inconsistent state, usually after a tool call was interrupted or a turn was edited mid-stream.

```
API Error: 400 due to tool use concurrency issues. Run /rewind to
recover the conversation.
API Error: 400 ... unexpected `tool_use_id` found in
`tool_result` blocks
API Error: 400 ... thinking blocks ... cannot be modified
```

All three variants mean the same thing: the sequence of `tool_use`, `tool_result`, and `thinking` blocks in history no longer matches what the API expects.

What to do:

- If you are using Opus 4.7 or Opus 4.8, run `claude update` first. Versions before v2.1.156 can trigger this error during normal tool use, and `/rewind` does not clear it.
- Run `/rewind`, or press Esc twice, to step back to a checkpoint before the corrupted turn and continue from there. See **Checkpointing** for how checkpoints are created and restored.

Usage Policy refusal

The API declined to respond because content in the conversation triggered a **Usage Policy** check. The message includes a Request ID you can quote to support if you believe the refusal is incorrect.

```
API Error: Claude Code is unable to respond to this request,
which appears to violate our Usage Policy
(https://www.anthropic.com/legal/aup). Please double press esc to
edit your last message or start a new session for Claude Code to
assist with a different task.
```

The check evaluates the full conversation, not only your latest prompt, so sending a new message in the same session usually re-triggers the same refusal. The same applies after exiting and reopening the session with `--continue` or `--resume`, since the transcript on disk still contains the triggering content.

What to do:

- Press Esc twice or run `/rewind` to step back to a checkpoint before the turn that triggered the refusal, then rephrase or take a different approach. See **Checkpointing**.
- If you cannot identify which turn caused it, run `/clear` to start a fresh conversation in the same project. Your previous conversation is preserved on disk and remains available in `/resume`.
- In **non-interactive mode** (`-p`), where rewind is unavailable, retry with a rephrased prompt in a new session without `--continue`. Policy checks vary by model, so switching to a different model with `--model` may also resolve the refusal in some cases.

Responses seem lower quality than usual

If Claude's answers seem less capable than you expect but no error is shown, the cause is usually conversation state rather than the model itself. Claude Code does not silently change model

versions. It can switch to a fallback model in three specific cases:

- A configured `--fallback-model` takes over after an availability error, for that turn only, with a notice in the transcript
- A Bedrock or Vertex AI startup check finds your default model unavailable
- **Automatic model fallback** on Fable 5 moves the session to the default Opus model and shows a notice in the transcript

The Model selection check below catches the second and third cases; the first appears as a transcript notice rather than a `/model` change. **Model configuration** explains when each fallback applies.

Check these first:

- **Model selection**: run `/model` to confirm you are on the model you expect. A previous `/model` choice or an `ANTHROPIC_MODEL` environment variable may have you on a smaller model than you intended.
- **Effort level**: run `/effort` to check the current reasoning level and raise it for hard debugging or design work. Defaults vary by model, so check before assuming you are below the maximum. See **Adjust effort level** for per-model defaults and the `ultrathink` shortcut.
- **Context pressure**: run `/context` to see how full the window is. If it is near capacity, run `/compact` at a natural breakpoint or `/clear` to start fresh. See **Explore the context window** for how auto-compact affects earlier turns.
- **Stale instructions**: large or outdated `CLAUDE.md` files and MCP tool definitions consume context and can steer responses. `/doctor` flags oversized memory files and subagent definitions; `/context` shows MCP tool token usage.

When a response goes wrong, rewinding usually works better than replying with corrections. Press Esc twice or run `/rewind` to step back to before the bad turn, then rephrase the prompt with more specifics. Correcting in-thread keeps the wrong attempt in context, which can anchor later answers to it. See **Checkpointing**.

If quality still seems off after checking the above, run `/feedback` and describe what you expected versus what you got. Feedback submitted this way includes the conversation transcript, which is the fastest way for Anthropic to diagnose a real regression. See **Report an error** if `/feedback` is unavailable in your environment.

Report an error

This page covers errors from the Claude API. For errors from other Claude Code components, see the relevant guide:

- MCP server failed to connect or authenticate: **MCP**
- Hook script failed or blocked a tool: **Debug hooks**
- Permission denied or filesystem errors during install: **Troubleshoot installation and login**

If an error is not listed here or the suggested fix does not help:

- Run `/feedback` inside Claude Code to send the transcript and a description to Anthropic. The command also offers to open a prefilled GitHub issue. On Bedrock, Vertex AI, Foundry, and other third-party providers, `/feedback` saves a local archive you can send to your Anthropic account representative instead.
- Run `/doctor` to check for local configuration problems
- Check **status.claude.com** for active incidents
- Search **existing issues** on GitHub